Báo cáo tuần

I. Xây dựng hàm build_targets (loss.py)

- Hàm build_targets có nhiệm vụ chuyển đổi các ground truth bounding box thành dạng tensor phù hợp để huấn luyện YOLO. Cụ thể, từ các box ở dạng tọa độ chuẩn hóa (x, y, w, h), hàm sẽ gán chúng vào các ô grid và anchor tương ứng ở từng scale.

```
def build_targets(targets, anchors, S, num_anchors_per_scale=3, ignore_iou_thresh=0.5):
```

- Đầu tiên, hàm khởi tạo một danh sách gồm ba tensor yolo_targets, tương ứng với ba scale của YOLO (13×13, 26×26, 52×52). Mỗi tensor có shape (B, 3, S, S, 6), trong đó B là batch size, 3 là số anchor trên mỗi scale, S là kích thước grid, và 6 tương ứng với các giá trị: objectness, x, y, w, h và class.

```
    yolo_targets = [
        torch.zeros((B, num_anchors_per_scale, S[i], S[i], 6), device=device)
        for i in range(3)
    ]
```

- Tiếp theo, danh sách anchor ban đầu (gồm 3 nhóm, mỗi nhóm 3 anchor) được reshape thành tensor có shape (9, 2). Việc này giúp dễ dàng so sánh tất cả anchor với một bounding box bằng cách tính IoU dựa trên width và height.

```
    anchors = torch.tensor(anchors, device = device).reshape(-1, 2) # (9, 2)
```

- Hàm sau đó duyệt qua từng ảnh trong batch, và trong mỗi ảnh, duyệt qua từng bounding box cùng nhãn class tương ứng. Với mỗi box, hàm tính IoU giữa kích thước của box đó và tất cả 9 anchor. Kết quả này được sắp xếp giảm dần để xác định anchor nào phù hợp nhất với box.

```python
for b in range(B):

    boxes = targets[b]['bboxes']
    labels = targets[b]['labels']
    for box, class_label in zip(boxes, labels):

        x, y, w, h = box

        iou_anchors = iou_width_height(
            torch.tensor([w, h], device=device),
            anchors
        )

        anchor_indices = iou_anchors.argsort(descending=True)
```

- Một biến cờ has_anchor được sử dụng để đảm bảo rằng mỗi object chỉ được gán cho một anchor chính (positive). Đây là nguyên tắc quan trọng trong YOLO: mỗi object chỉ có một anchor "chịu trách nhiệm".

- Sau đó, hàm duyệt qua các anchor theo thứ tự từ IoU cao xuống thấp. Với mỗi anchor, nó xác định anchor đó thuộc scale nào và vị trí của nó trong scale đó. Đồng thời, tọa độ của bounding box được chuyển từ hệ tọa độ ảnh sang hệ tọa độ grid bằng cách nhân với S và lấy phần nguyên để xác định cell (i, j).

```python
        has_anchor = False

        for anchor_idx in anchor_indices:

            scale_idx = anchor_idx // num_anchors_per_scale
            anchor_on_scale = anchor_idx % num_anchors_per_scale

            S_scale = S[scale_idx]
            # cell chịu trách nhiệm cho box ở grid space
            i = int(S_scale * y)
            j = int(S_scale * x)
```

- Tiếp theo, hàm kiểm tra xem vị trí (cell, anchor) đó đã được gán object chưa. Nếu chưa và object này chưa có anchor nào được gán, thì anchor hiện tại sẽ trở thành positive anchor. Khi đó, objectness được gán bằng 1, và tọa độ box được chuyển sang dạng offset trong cell. Cụ thể, x và y được tính lại thành phần thập phân trong cell, còn w và h được scale theo kích thước grid. Nhãn class cũng được lưu vào tensor target.

- Sau khi gán xong anchor chính, biến has_anchor được đặt thành True để ngăn không cho các anchor khác tiếp tục được gán là positive cho cùng object đó.

```python
anchor_taken = yolo_targets[scale_idx][b, anchor_on_scale, i, j, 0]

if not anchor_taken and not has_anchor:

    yolo_targets[scale_idx][b, anchor_on_scale, i, j, 0] = 1

    x_cell = S_scale * x - j
    y_cell = S_scale * y - i

    w_cell = w * S_scale
    h_cell = h * S_scale

    yolo_targets[scale_idx][b, anchor_on_scale, i, j, 1:5] = \
        torch.tensor([x_cell, y_cell, w_cell, h_cell], device=device)

    yolo_targets[scale_idx][b, anchor_on_scale, i, j, 5] = class_label

    has_anchor = True
```

- Trong trường hợp một anchor không phải là anchor tốt nhất nhưng vẫn có IoU lớn hơn ngưỡng ignore_iou_thresh, thì anchor đó sẽ được gán giá trị objectness là -1. Điều này có nghĩa là anchor này sẽ bị bỏ qua trong quá trình tính loss, không bị coi là positive cũng không bị coi là negative. Cách xử lý này giúp tránh việc phạt những anchor "gần đúng" một cách không cần thiết.

```python
elif not anchor_taken and iou_anchors[anchor_idx] > ignore_iou_thresh:

    yolo_targets[scale_idx][b, anchor_on_scale, i, j, 0] = -1
```

- Kết quả cuối cùng của hàm là một danh sách gồm ba tensor target, tương ứng với ba scale, trong đó mỗi object được gán vào đúng một anchor tốt nhất, các anchor tương tự được ignore, và các anchor còn lại được coi là background.

II. Xây dựng hàm intersection over union (utils.py)

1. Hàm iou_width_height

Hàm này dùng để tính IoU nhưng **chỉ dựa trên width và height**, không xét vị trí (x, y). Vì vậy nó không phải IoU "thật", mà là một dạng **IoU xấp xỉ theo kích thước**.

Cụ thể, đầu vào boxes1 và boxes2 có dạng: (w, h)

Hàm giả định hai box này **có cùng tâm**, nên phần giao nhau chỉ phụ thuộc vào kích thước.

Phần intersection được tính bằng cách lấy chiều rộng nhỏ hơn và chiều cao nhỏ hơn:

$$\text{intersection} = \min(w1, w2) * \min(h1, h2)$$

Điều này tương đương với việc đặt hai box chồng lên nhau hoàn toàn ở tâm, thì phần overlap sẽ là phần nhỏ hơn theo từng chiều.

Tiếp theo, union được tính theo công thức:

union = area1 + area2 - intersection

trong đó:

- area1 = w1 × h1
- area2 = w2 × h2

Cuối cùng, IoU được tính:

IoU = intersection / union

Hàm này thường được dùng trong YOLO để:

- so sánh ground truth box với anchor box
- chọn anchor nào có kích thước phù hợp nhất

Hàm này **không quan tâm vị trí**, chỉ quan tâm hình dạng (aspect ratio và scale).

2. Hàm intersection_over_union

Đây là hàm tính IoU "đúng nghĩa", xét cả vị trí và kích thước của bounding box.

Hàm hỗ trợ hai định dạng box:

- "xywh": (x_center, y_center, width, height)
- "xyxy": (x1, y1, x2, y2)

Bước 1: Chuyển về dạng (x1, y1, x2, y2)

Nếu input là "xywh", hàm sẽ chuyển sang dạng góc:

$$x1 = x - \frac{w}{2}, y1 = y - \frac{h}{2}$$
$$x2 = x + \frac{w}{2}, y2 = y + \frac{h}{2}$$

Việc này giúp dễ tính phần giao nhau vì chỉ cần so sánh biên.

Bước 2: Tính vùng giao nhau (intersection)

Hàm tìm phần overlap giữa hai box bằng cách:

- lấy cạnh trái lớn hơn (max)
- lấy cạnh phải nhỏ hơn (min)

$$x_{left} = \max(x1_1, x1_2)$$
$$x_{right} = \min(x2_1, x2_2)$$

Tương tự cho y.

Diện tích giao nhau:

$$intersection = (x_{right} - x_{left}) \times (y_{bottom} - y_{top})$$

Phần .clamp(0) đảm bảo nếu hai box không giao nhau thì intersection = 0 (không bị âm).

Bước 3: Tính diện tích từng box

$$area = (x2 - x1) \times (y2 - y1)$$

Việc dùng .clamp(0) giúp tránh lỗi nếu box bị sai (x2 < x1).

Bước 4: Tính IoU

$$IoU = \frac{intersection}{area1 + area2 - intersection}$$

Phần 1e-6 được thêm vào mẫu số để tránh chia cho 0.

III. Xây dựng model YOLO v3 (model.py)

Đoạn code xây dựng YOLOv3 theo đúng kiến trúc gồm một backbone để trích xuất đặc trưng và các detection head để dự đoán bounding box ở nhiều tỉ lệ khác nhau. Phần backbone Darknet53 nhận ảnh đầu vào và tạo ra ba feature map ở các mức độ khác nhau về độ phân giải, thường tương ứng với kích thước 52×52, 26×26 và 13×13. Ba feature này lần lượt được lưu vào route1,

route2, route3, trong đó route3 là đặc trưng sâu nhất (semantic mạnh nhất nhưng độ phân giải thấp nhất), còn route1 là đặc trưng nông nhất (giữ nhiều thông tin vị trí).

Ở bước đầu tiên trong forward, route3 được đưa vào head1 để tạo ra dự đoán ở scale 13×13. Detection block này vừa xử lý feature vừa xuất ra hai thành phần: một feature trung gian (route) dùng cho FPN và một tensor dự đoán (out1). Feature route này sau đó được đưa qua một convolution 1×1 để giảm số kênh, rồi upsample lên gấp đôi kích thước không gian. Việc upsample giúp đưa feature từ mức 13×13 lên 26×26 để có thể kết hợp với route2, là feature ở cùng kích thước. Hai feature này được nối (concatenate) theo chiều channel, tạo thành một tensor giàu thông tin hơn, vừa có ngữ nghĩa sâu từ tầng trên vừa có chi tiết không gian từ tầng dưới.

```
route1, route2, route3 = self.backbone(x)

# Scale 1
route, out1 = self.head1(route3)
```

Tensor sau khi concat được đưa vào head2 để tạo dự đoán ở scale 26×26. Tương tự như trước, head2 cũng trả về một feature trung gian (route) và một output (out2). Feature trung gian này tiếp tục được xử lý qua convolution 1×1 để giảm channel, sau đó upsample lên 52×52 và nối với route1, là feature có độ phân giải cao nhất từ backbone. Điều này cho phép kết hợp thông tin đa tỉ lệ một lần nữa, đặc biệt hữu ích cho việc phát hiện các vật thể nhỏ.

```
# Scale 2
x = self.conv_for_P4(route)
x = self.upsample(x)
x = torch.cat([x, route2], dim=1)
route, out2 = self.head2(x)
```

Cuối cùng, tensor kết hợp ở scale 52×52 được đưa vào head3 để tạo ra dự đoán cuối cùng (out3). Ở scale này không cần trả lại feature trung gian nữa vì đây là tầng cuối. Kết quả của toàn bộ mạng là ba tensor out1, out2, out3, tương ứng với ba mức tỉ lệ khác nhau. Mỗi tensor chứa các dự đoán bounding box, objectness và class cho từng cell và từng anchor.

```
# Scale 3
x = self.conv_for_P3(route)
x = self.upsample(x)
x = torch.cat([x, route1], dim=1)
_, out3 = self.head3(x)

return out1, out2, out3
```

Toàn bộ quá trình này thể hiện rõ ý tưởng của YOLOv3: sử dụng Feature Pyramid Network để kết hợp thông tin từ nhiều mức độ trừu tượng khác nhau, từ đó giúp mô hình phát hiện tốt cả vật thể lớn, vừa và nhỏ trong cùng một ảnh.